

C Process API

OSTEP 05

Sam Ogden

Updated 2026-06-01 15:20

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

C Process API

- Sam Ogden (slides by Glenn Brunns)
- CSUMB

Lecture Objectives

- At the end of this lecture, you should be able to:
- List the functions in the C process API
 - `fork()`
 - `wait()`
 - `exec(...)`
- Describe what the functions do
- Write C code that uses the functions

Note that the “functions” are often families of functions

Process control in C

- Sometimes applications want to create or kill processes.
- A running app is a process – so sometimes processes want to create or kill processes.
- For example, the shell will create and kill processes.

```
1 while (true) {  
2     print "$"  
3     get command from user  
4     start command in new process  
5     wait for process to complete  
6 }
```

Creating new
processes: `fork()`

How do we create a new process?

NAME [top](#)

`fork` - create a child process

SYNOPSIS [top](#)

```
#include <unistd.h>
```

```
pid_t fork(void);
```

DESCRIPTION [top](#)

fork() creates a new process by duplicating the calling process. The new process is referred to as the *child* process. The calling process is referred to as the *parent* process.

The child process and the parent process run in separate memory spaces. At the time of **fork()** both memory spaces have the same content. Memory writes, file mappings ([mmap\(2\)](#)), and unmappings ([munmap\(2\)](#)) performed by one of the processes do not affect the other.

The child process is an exact duplicate of the parent process except for the following points:

- * The child has its own unique process ID, and this PID does not match the ID of any existing process group ([setpgid\(2\)](#)) or session.
- * The child's parent process ID is the same as the parent's process ID.

Process creation with fork

```
1 int main() {
2     int rc = fork();
3     if ( rc == 0 ) {
4         // If RC == 0, we are the child thread
5         printf("I am the child thread.  child pid = %d\n", getpid());
6     } else if (rc > 0) {
7         // If rc > 0, we are the parent thread and rc is the child's PID
8         printf("I am the parent thread.  child pid = %d\n", rc);
9     } else {
10        // If rc < 0, something went wrong
11        fprintf(stderr, "Fork failed");
12        exit(8);
13    }
14 }
```

- Fork is weird and wonderful!
- It *clones* the process that called fork()
 - The memory space is duplicated!
- One process makes the call; two processes see the return!

A process tree

- Process A created two child processes: B and C
- Process B is a “child process” of A
- Process D is a “child process” of B

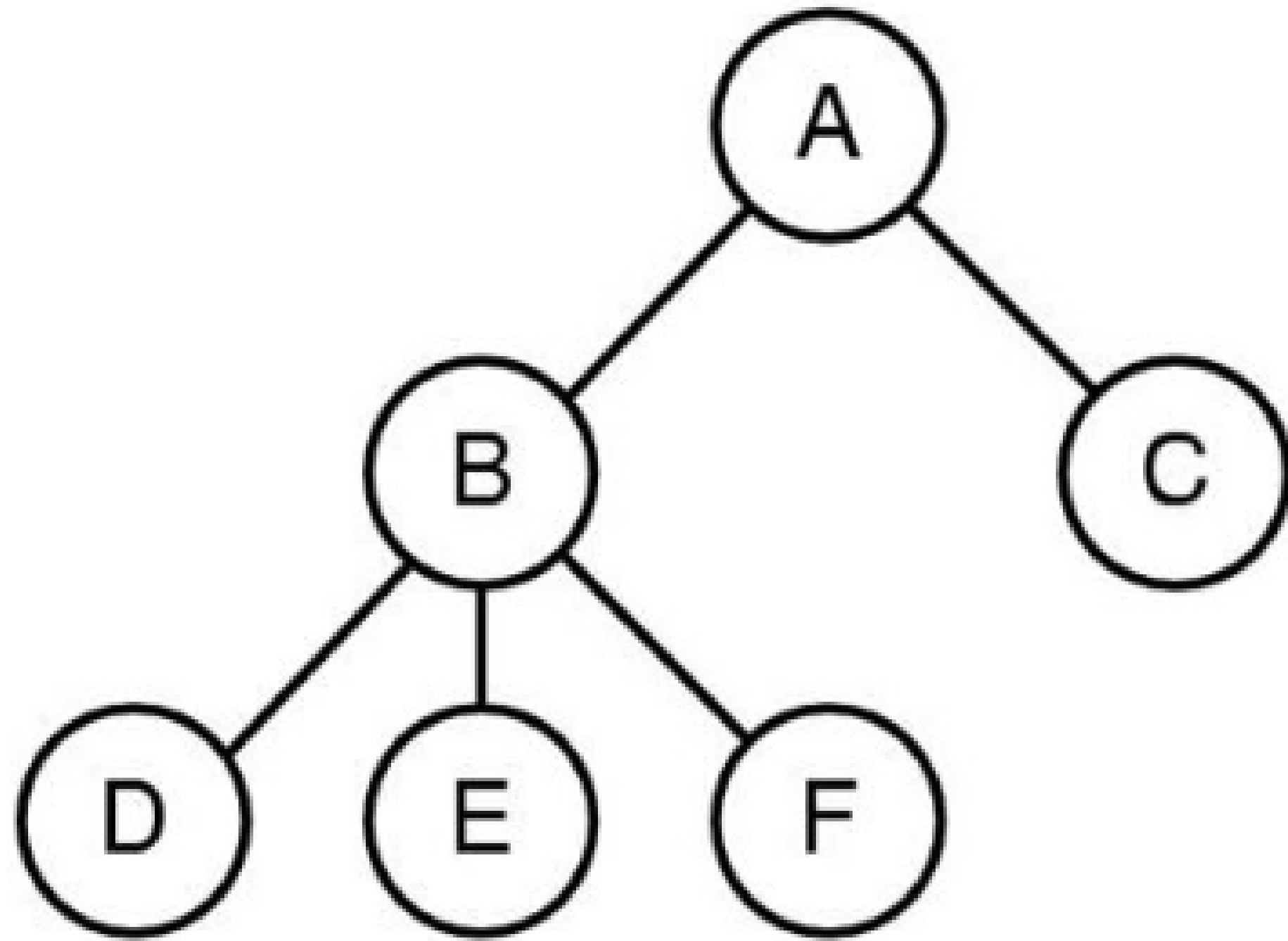


Figure from Tanenbaum, Modern Operating Systems

Exercise: what does this output?

```
1 int main(int argc, char* argv[]) {  
2     fork();  
3     printf("hello!\n");  
4     return 0;  
5 }
```

```
1 $ ./fork-prob4  
2 hello!  
3  
4 $ hello!
```

Exercise: what is wrong with this?

```
1 int main(int argc, char* argv[])
2 {
3     int pid = getpid();
4     int rc = fork();
5     if (rc < 0) {
6         fprintf(stderr, "fork failed\n");
7         exit(1);
8     } else if (rc == pid) {
9         printf("child (pid:%d)\n", (int)getpid());
10    } else {
11        printf("parent (pid:%d)\n", (int)getpid());
12    }
13    return 0;
14 }
```

We're asserting that the PID hasn't changed in the child, but it will have!

Exercise: what does this output?

```
1 int main(int argc, char* argv[]) {  
2     fork();  
3     fork();  
4     printf("hello!\n");  
5     return 0;  
6 }
```

```
1  
2 $ ./fork-prob5  
3 hello!  
4  
5 $ hello!  
6 hello!  
7 hello!
```

Coordinating
between
processes: `wait()`

Wait

```
1
2 int main() {
3     int rc = fork();
4     if ( rc == 0 ) {
5         printf("I am the child process\n");
6     } else if ( rc > 0 ) {
7         int wc = wait(NULL);
8         printf("I am the parent process.\n");
9         printf("The child exited with status code wc = %d\n", wc);
10    } else {
11        fprintf(stderr, "Fork failed.\n");
12        exit(8);
13    }
14 }
```

A wait call **blocks** until one of the children of the calling process has terminated. - If there are no children it returns immediately

Wait man page

NAME [top](#)

wait, waitpid, waitid - wait for process to change state

SYNOPSIS [top](#)

```
#include <sys/wait.h>

pid_t wait(int *wstatus);
pid_t waitpid(pid_t pid, int *wstatus, int options);

int waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options);
/* This is the glibc and POSIX interface; see
   NOTES for information on the raw system call. */
```

Feature Test Macro Requirements for glibc (see
[feature_test_macros\(7\)](#)):

```
waitid():
    Since glibc 2.26:
        _XOPEN_SOURCE >= 500 || _POSIX_C_SOURCE >= 200809L
    Glibc 2.25 and earlier:
        _XOPEN_SOURCE
        || /* Since glibc 2.12: */ _POSIX_C_SOURCE >= 200809L
        || /* Glibc <= 2.19: */ _BSD_SOURCE
```

DESCRIPTION [top](#)

All of these system calls are used to wait for state changes in a child of the calling process, and obtain information about the child whose state has changed. A state change is considered to be: the child terminated; the child was stopped by a signal; or the child was resumed by a signal. In the case of a terminated child, performing a wait allows the system to release the resources associated with the child; if a wait is not performed, then the terminated child remains in a "zombie" state (see NOTES below).

Exercise: what does this output?

```
1 int main() {
2     printf("pid = %d\n", (int)getpid());
3     int rc = fork();
4     if (rc < 0) {
5         fprintf(stderr, "fork failed\n");
6         exit(1);
7     } else {
8         wait(NULL);
9     }
10    printf("pid = %d\n", (int)getpid());
11    return 0;
12 }
```

```
1 $ ./wait-prob1
2 pid = 2
3 pid = 3
4 pid = 2
```

(Note that the numbers may be different if you try this code!)

Running other
code: `exec()`

Exec

```
1 int main() {  
2     printf("This text will be printed.\n");  
3  
4     execlp("ls", "ls", "-l", NULL);  
5  
6     fprintf(stderr, "exec failed\n");  
7     return 1;  
8 }
```

`exec` call changes the code in a process

Note

In the above, we will not make it to line 5 unless `exec` fails!

exec man page

NAME [top](#)

`execl, execlp, execl, execv, execvp, execvpe` - execute a file

SYNOPSIS [top](#)

```
#include <unistd.h>

extern char **environ;

int execl(const char *pathname, const char *arg, ...
          /*, (char *) NULL */);
int execlp(const char *file, const char *arg, ...
          /*, (char *) NULL */);
int execl(const char *pathname, const char *arg, ...
          /*, (char *) NULL, char *const envp[] */);
int execv(const char *pathname, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);
```

Feature Test Macro Requirements for glibc (see [feature_test_macros\(7\)](#)):

```
execvpe():
    _GNU_SOURCE
```

DESCRIPTION [top](#)

The **exec()** family of functions replaces the current process image with a new process image. The functions described in this manual page are layered on top of [execve\(2\)](#). (See the manual page for [execve\(2\)](#) for further details about the replacement of the current process image.)

Summary

- `fork` – make a copy of the current process
- `wait` – wait for a child process to terminate
 - `wait(...)`: wait for *any* child
 - `waitpid(...)`: wait for a *specific* child
- `exec` – code of current process is replaced
 - `exec1*(...)`: execute a *specific* program + options
 - `execv*(...)`: execute *variable* program + options